ICS 202 – Data Structures and Algorithms

Topic 12: String Matching

Question bank: 40 multiple choice + 10 programming

Questions from ICS 202 old exams, quizzes and activities (AI_Work). **[new]** marks new questions written in the same style.

Part A: Multiple choice

All questions use the course conventions: the brute-force algorithm compares the pattern with the text from left to right and shifts the pattern one position after a mismatch; the KMP `next` array has one entry per pattern index, `next[0] = -1`, and `next[j]` is the length of the border (longest proper prefix that is also a proper suffix) of `P[0..j-1]`. A "comparison" is one comparison of a text character with a pattern character.

Q1. [new]

The brute-force algorithm searches for the pattern `ABAB` in the text `ABACABAABCAB`. How many character comparisons are performed in total?

(A) 12

(B) 16

(C) 19

(D) 36

Answer: (C). Alignments 0..8 cost 4,1,2,1,4,1,2,3,1 comparisons; the pattern never occurs, so all 9 alignments are tried: 19.

My answer

Q2. [new]

Using the brute-force algorithm to find the first occurrence of `SIMPLE` in the text `THIS IS A SIMPLE EXAMPLE` (spaces are characters, indexes start at 0), at which index is the pattern found and how many comparisons are made?

(A) Index 10, 18 comparisons

(B) Index 10, 16 comparisons

(C) Index 9, 18 comparisons

(D) Index 11, 21 comparisons

Answer: (A). Alignments 0..9 each cost 1 comparison except alignment 3 (`S` matches, then space vs `I`: 2) and alignment 6 (2); that is 12, plus 6 at index 10: 18.

My answer

Q3.

Activity 12-01 (T252)

Let `T = ACGACACATAGTCACTTGGCA` and the pattern `P = ACATA`. How many comparisons does the brute-force string matching algorithm perform to find the pattern in the text?

(A) 14

(B) 15

(C) 17

(D) 21

Answer: (B). Alignments 0..5 cost 3, 1, 1, 4, 1 and 5 (match at index 5): 15 (old key said 14; it counted only 4 for the final full match).

My answer

Q4.

Quiz 5 Ver A key (T251)

Consider searching for the pattern `bacbaaa` in the text `bacbacbaaacbac`. Using the brute-force search method, the total number of character comparisons needed before finding the pattern in the text equals

(A) 8

(B) 10

(C) 13

(D) 15

Answer: (D). Alignment 0 costs 6 (mismatch at `c/a`), alignments 1 and 2 cost 1 each, alignment 3 is a full match of 7: 15.

My answer

Q5.

[new]

The brute-force algorithm searches for `P = AAAB` in `T = AAAAAAAAAA` (ten `A`'s). How many comparisons are made?

(A) 7

(B) 10

(C) 28

(D) 40

Answer: (C). There are $|T| - |P| + 1 = 7$ alignments and each one matches `AAA` then fails on `B`: $7 \times 4 = 28$ (the worst case).

My answer

Q6. [new]

In the brute-force (naive) algorithm, what happens when a mismatch occurs between `T[i]` and `P[j]`?

(A) The pattern is shifted one position to the right and the comparison restarts from `P[0]`, so the text index moves back.

(B) The pattern is shifted $j - \texttt{next}[j]$ positions to the right and the text index does not move back.

(C) The pattern is shifted $|P|$ positions to the right.

(D) The search stops and returns -1.

Answer: (A). The naive algorithm always shifts by 1 and restarts comparing from the first pattern character (overlapping shift by 1).

My answer

Q7. [new]

The worst-case number of comparisons of the brute-force algorithm for a text T and a pattern P is

(A) $|T| - |P| + 1$, i.e. $O(|T|)$

(B) $(|T| - |P| + 1) \cdot |P|$, i.e. $O(|T|\,|P|)$

(C) $|T| + |P|$, i.e. $O(|T| + |P|)$

(D) $|P| \log |T|$, i.e. $O(|P| \log |T|)$

Answer: (B). The outer loop runs $|T|-|P|+1$ times and the inner loop runs up to $|P|$ times; $|T|-|P|+1$ is the best case.

My answer

Q8. [new]

KMP searches for `abababa` in the text `ababcdabbabababab`. The first mismatch happens at text index $i = 4$ (`c`) and pattern index $j = 4$. Where does the search resume?

(A) $i = 1$, $j = 0$

(B) $i = 4$, $j = 0$

(C) $i = 4$, $j = 2$

(D) $i = 5$, $j = 0$

Answer: (C). `next` for `abababa` is $-1, 0, 0, 1, 2, 3, 4$; `next[4] = 2` (border `ab` of `abab`) and i is not moved back.

My answer

Q9. [new]

Which statement about the KMP algorithm is TRUE?

(A) Like the brute-force algorithm, it moves the text index i back after every mismatch.

(B) It never moves the text index i back; on a mismatch only the pattern index j changes (to `next[j]`).

(C) It preprocesses the text, not the pattern.

(D) It can only find non-overlapping occurrences of a pattern.

Answer: (B). KMP preprocesses the pattern (the `next` array) and does not reset i; it can be used for overlapping and non-overlapping searches.

My answer

Q10. [new]

Counting the empty string λ, how many proper prefixes does the string `ESTATES` have?

(A) 6

(B) 7

(C) 8

(D) 14

Answer: (B). λ, `E`, `ES`, `EST`, `ESTA`, `ESTAT`, `ESTATE`: a string of length n has n proper prefixes (including λ).

My answer

Q11. Activity 12-01 (T252)

What is the border of the string `BABABACBBA`?

(A) λ (empty)

(B) `B`

(C) `BA`

(D) `BAB`

Answer: (C). `BA` is both a proper prefix and a proper suffix; `BAB` is not a suffix (`CBBA` ends in `BBA`).

My answer

Q12. [new]

What is the border of the string `BABABABA`?

(A) `BA`

(B) `BABA`

(C) `BABABA`

(D) `BABABAB`

Answer: (C). `BABABA` is the longest proper prefix that is also a proper suffix (`BABABAB` is not a suffix).

My answer

Q13. [new]

What is the border of the string `ABCAABC`?

(A) `A`

(B) `AB`

(C) `ABC`

(D) `ABCA`

Answer: (C). `ABC` is a proper prefix and a proper suffix; `ABCA` is not a suffix.

My answer

Q14. [new]

In the textbook version of KMP used in the course, `next[j]` (for $j \geq 1$) is defined as

(A) the length of the border of `P[0..j]`

(B) the length of the border of `P[0..j-1]`

(C) the number of times `P[j]` occurs in `P[0..j-1]`

(D) the index of the last occurrence of `P[j]` in the pattern

Answer: (B). `next[j]` is the border length of the matched part `P[0..j-1]`, with `next[0] = -1` as a sentinel.

My answer

Q15. Old exam (String Matching)

Find the KMP `next` (lps) array for the pattern `AABAABC`.

(A) $-1, 0, 1, 0, 1, 2, 3$

(B) $-1, 0, 1, 0, 1, 2, 0$

(C) $-1, 0, 0, 0, 1, 2, 3$

(D) $0, 1, 0, 1, 2, 3, 0$

Answer: (A). Borders of `A`, `AA`, `AAB`, `AABA`, `AABAA`, `AABAAB` have lengths 0,1,0,1,2,3.

My answer

Q16. Old exam (String Matching)

Find the `next` array of the Knuth–Morris–Pratt algorithm for the pattern `ONIONS`.

(A) $-1, 0, 0, 0, 1, 2$

(B) $-1, 0, 0, 1, 2, 0$

(C) $-1, 0, 0, 0, 1, 0$

(D) $-1, 0, 1, 0, 1, 2$

Answer: (A). `ONIO` has border `O` (1) and `ONION` has border `ON` (2); the others have empty borders.

My answer

Q17. Activity 12-01 (T252)

Create the lps (`next`) array for the pattern `AACAAABA`.

(A) $-1, 0, 1, 0, 1, 2, 3, 0$

(B) $-1, 0, 1, 0, 1, 2, 2, 0$

(C) $-1, 0, 1, 0, 1, 2, 2, 1$

(D) $-1, 0, 0, 0, 1, 2, 2, 0$

Answer: (B). `AACAAA` has border `AA` (2), not `AAC`; `AACAAAB` has border λ.

My answer

Q18. Activity 12-01 (T252)

Create the lps (`next`) array for the pattern `AAAABAACD`.

(A) $-1, 0, 1, 2, 3, 0, 1, 2, 3$

(B) $-1, 0, 1, 2, 3, 4, 1, 2, 0$

(C) $-1, 0, 1, 2, 3, 0, 1, 2, 0$

(D) $-1, 0, 1, 2, 3, 0, 1, 1, 0$

Answer: (C). `AAAABA` has border `A` (1), `AAAABAA` has border `AA` (2), `AAAABAAC` has none (0).

My answer

Q19. Quiz 5 V1 key (T251)

Which represents the KMP `next` array (lps) of the pattern `KKKKCDKK`?

(A) $\langle -1, 0, 1, 2, 3, 0, 0, 1 \rangle$

(B) $\langle 0, 1, 2, 3, 4, 0, 1, 2 \rangle$

(C) $\langle -1, 0, 1, 2, 3, 0, 1, 2 \rangle$

(D) $\langle -1, 0, 2, 3, 4, 0, 1, 2 \rangle$

Answer: (A). `KKKKC` and `KKKKCD` have empty borders, and `KKKKCDK` has border `K` (1).

My answer

Q20. Quiz 5 key (T232)

In the KMP algorithm for searching for a pattern P in a text T, the `next` array for the pattern `aaaaba` is

(A) $-1, 0, 1, 2, 3, 1$

(B) $-1, 0, 1, 2, 3, 0$

(C) $-1, 0, 1, 2, 3, 4$

(D) $0, 1, 2, 3, 0, 1$

Answer: (B). `a, aa, aaa, aaaa` give 0,1,2,3 and `aaaab` has empty border.

My answer

Q21. Quiz 5 Ver A key (T251)

Determine the `next` array for the pattern `AAABAAA`.

(A) $-1, 0, 1, 2, 0, 1, 2$

(B) $-1, 0, 1, 2, 0, 1, 1$

(C) $-1, 0, 1, 2, 0, 1, 2, 3$

(D) $-1, 0, 0, 1, 0, 1, 2$

Answer: (A). `AAAB` has border λ, `AAABA` has `A`, `AAABAA` has `AA`.

My answer

Q22. Quiz 5 Ver A key (T242)

Find the `next` array (lps-array) for the pattern `ACACAGT`.

(A) $-1, 0, 0, 1, 2, 3, 0$

(B) $-1, 0, 0, 1, 2, 0, 0$

(C) $-1, 0, 1, 2, 3, 0, 0$

(D) $-1, 0, 0, 1, 2, 3, 4$

Answer: (A). `ACA`: 1, `ACAC`: 2, `ACACA`: 3 (border `ACA`), `ACACAG`: 0.

My answer

Q23. [new]

Find the `next` array for the pattern `AABCADAABE`.

(A) $-1, 0, 1, 0, 0, 1, 0, 1, 2, 3$

(B) $-1, 0, 1, 0, 0, 1, 0, 1, 2, 0$

(C) $-1, 0, 1, 0, 1, 1, 0, 1, 2, 3$

(D) $-1, 0, 0, 0, 0, 1, 0, 1, 2, 3$

Answer: (A). `AABCA`: 1, `AABCAD`: 0, `AABCADA`: 1, `AABCADAA`: 2, `AABCADAAB`: 3 (border `AAB`).

My answer

Q24. [new]

Find the `next` array for the pattern `AAACAAAAAC`.

(A) $-1, 0, 1, 2, 0, 1, 2, 3, 4, 5$

(B) $-1, 0, 1, 2, 0, 1, 2, 3, 3, 3$

(C) $-1, 0, 1, 2, 0, 1, 2, 3, 0, 0$

(D) $-1, 0, 1, 2, 0, 1, 2, 3, 3, 4$

Answer: (B). `AAACAAA`: 3; `AAACAAAA` and `AAACAAAAA` still have border `AAA` only (3), since `AAAC` is not a suffix of them.

My answer

Q25. [new]

Find the `next` array for the pattern `AABAACAABAA`.

(A) $-1, 0, 0, 0, 0, 1, 2, 0, 1, 2, 3$

(B) $-1, 0, 1, 0, 1, 2, 0, 1, 2, 3, 3$

(C) $-1, 0, 1, 0, 1, 2, 0, 1, 2, 3, 4$

(D) $-1, 0, 1, 0, 1, 2, 0, 1, 2, 3, 5$

Answer: (C). After `AABAAC` (border 0) the prefix is repeated: `AABAACA`: 1, . . . , `AABAACAABA`: 4 (border `AABA`).

My answer

Q26. [new]

The pattern `AABAACAABAC` has `next` array $-1, 0, 1, 0, 1, 2, 0, 1, 2, 3, 4$. During a KMP search a mismatch occurs at pattern index $j = 9$. By how many positions is the pattern shifted, and at which pattern index does the search resume?

(A) Shift 1, resume at $j = 0$

(B) Shift 3, resume at $j = 6$

(C) Shift 6, resume at $j = 3$

(D) Shift 9, resume at $j = 0$

Answer: (C). Shift $j - \texttt{next}[j] = 9 - 3 = 6$ and resume at $\max\{0, \texttt{next}[9]\} = 3$.

My answer

Q27. [new]

In KMP, if a mismatch occurs at pattern index $j = 0$, then

(A) the pattern is shifted by $|P|$ and the search resumes at $j = 0$

(B) the pattern is shifted by 1 and the search resumes at $j = 0$

(C) the pattern is not shifted and the search resumes at $j = -1$ with the same text character

(D) the search ends because the text index cannot move

Answer: (B). The shift is $0 - (-1) = 1$ and the search resumes at $\max\{0, -1\} = 0$; in the code j becomes -1 and then $i{+}{+}$, $j{+}{+}$.

My answer

Q28. Old exam (String Matching)

The text is `ONIONIONSPL` and the pattern is `ONIONS` (`next` $= -1, 0, 0, 0, 1, 2$). Starting with `text[0]` aligned with `pattern[0]`, a mismatch occurs at the last character of the pattern, i.e. at text index $i = 5$ and pattern index $j = 5$. Using KMP, at which values of i and j does the search resume?

(A) $i = 1$, $j = 0$

(B) $i = 5$, $j = 0$

(C) $i = 5$, $j = 2$

(D) $i = 6$, $j = 2$

Answer: (C). i stays at 5 and $j = \texttt{next}[5] = 2$ (border `ON` of `ONION`).

My answer

Q29. [new]

Trace KMP searching for the pattern `ABABX` (`next` $= -1, 0, 0, 1, 2$) in the text `ABTABABYABABXKABABXZM` to find its first occurrence. How many comparisons are made in total?

(A) 14

(B) 16

(C) 18

(D) 21

Answer: (B). Steps cost 3, 1, 5, 1, 1, 5 comparisons; the pattern is found at index 8.

My answer

Q30. [new]

Using the pattern `AABAAC` (`next` $= -1, 0, 1, 0, 1, 2$), KMP searches the text `AACAABAABAACAACB`. What does the search return, and how many comparisons are made?

(A) Returns 6 after 15 comparisons

(B) Returns 6 after 13 comparisons

(C) Returns 3 after 15 comparisons

(D) Returns 9 after 19 comparisons

Answer: (A). Steps cost 3, 1, 1, 6, 4 comparisons; the last alignment starts at index 6.

My answer

Q31. [new]

Given the pattern `AACAAABA` (`next` $= -1, 0, 1, 0, 1, 2, 2, 0$), KMP searches the text `AACAACAAABAY`. How many comparisons are made until the pattern is found?

(A) 10

(B) 12

(C) 14

(D) 17

Answer: (B). The first alignment fails at $j = 5$ after 6 comparisons; shift $5 - 2 = 3$, resume at $j = 2$ and match the remaining 6 characters: 12; found at index 3.

My answer

Q32. [new]

Given the pattern `ABXABYABXZ` (`next` $= -1, 0, 0, 0, 1, 2, 0, 1, 2, 3$),
KMP searches the text `ABXABYABXABYABX`. What is the result?

(A) Found at index 6 after 16 comparisons

(B) Not found (-1) after 16 comparisons

(C) Not found (-1) after 15 comparisons

(D) Not found (-1) after 25 comparisons

Answer: (B). 10 comparisons fail at `Z`; shift 6, resume at $j = 3$ and match `ABYABX` (6 more) until the text ends: 16, no occurrence.

My answer

Q33. Activity 12-02 (T252)

Let `T = ABABABABABABABABABC` and the pattern `P = ABABABC` (`next` $= -1, 0, 0, 1, 2, 3, 4$). How many comparisons does the KMP algorithm perform?

(A) 19

(B) 25

(C) 43

(D) 49

Answer: (B). The first alignment costs 7 (mismatch at $j = 6$); each later alignment resumes at $j = \texttt{next}[6] = 4$ and costs 3. Alignments at 2, 4, 6, 8, 10 fail and the one at 12 matches: $7 + 6 \times 3 = 25$ (the activity sheet counted 7 per alignment).

My answer

Q34. Quiz 5 V1 key (T251)

Run KMP to search for the pattern `ABXABYABXZ` (`next` $= -1, 0, 0, 0, 1, 2, 0, 1, 2, 3$)
in the text `ABXABYABXABXABYABXZ`. What is the total number of comparisons?

(A) 19

(B) 21

(C) 23

(D) 28

Answer: (B). Iterations cost 10 (shift 6), 3 (shift 3) and 8 (match at index 9): 21.

My answer

Q35.

Quiz 5 Ver A key (T251)

Consider searching for the pattern `bacbaaa` in the text `bacbacbaaacbac`. Using the KMP algorithm, the total number of character comparisons needed before finding the pattern in the text equals

(A) 8

(B) 11

(C) 13

(D) 15

Answer: (B). `next` $= -1, 0, 0, 0, 1, 2, 0$. The first alignment fails at $j = 5$ after 6 comparisons; shift 3, resume at $j = 2$ and match 5 more: 11.

My answer

Q36.

[new]

KMP searches for `P = AAAB` (`next` $= -1, 0, 1, 2$) in `T = AAAAAAAAAA` (ten `A`'s). How many comparisons are made?

(A) 10

(B) 17

(C) 28

(D) 40

Answer: (B). The first alignment costs 4; the next 6 alignments resume at $j = \texttt{next}[3] = 2$ and cost 2 each; the last alignment makes 1 comparison before the text ends: $4 + 6 \times 2 + 1 = 17$ (brute force needs 28).

My answer

Q37.

[new]

The pattern `ABA` is searched for in the text `ABABABA`. At which text indexes is it found using an overlapping search and using a non-overlapping search?

(A) Overlapping: 0, 2, 4; non-overlapping: 0, 4

(B) Overlapping: 0, 4; non-overlapping: 0, 2, 4

(C) Overlapping: 0, 2, 4; non-overlapping: 0, 2

(D) Overlapping: 0, 2; non-overlapping: 0, 4

Answer: (A). Overlapping occurrences may share characters; a non-overlapping search continues after the end of a match (shift $|P|$), so index 2 is skipped.

My answer

Q38. [new]

What does `kmpSearch("ABABDABACDABABCABAB", "ABABCABAB")` return? (`findNext` fills `next` as defined in the course.)

```
int kmpSearch(String T, String P) {
    int[] next = findNext(P);
    int i = 0, j = 0;
    while (i < T.length() && j < P.length()) {
        if (j == -1 || T.charAt(i) == P.charAt(j)) {
            i++; j++;
        } else
            j = next[j];
    }
    if (j == P.length()) return i - j;
    else return -1;
}
```

(A) -1

(B) 9

(C) 10

(D) 19

Answer: (C). The loop ends with $i = 19$, $j = 9$, so it returns $19 - 9 = 10$, the start of the match.

My answer

Q39. [new]

What is the time complexity of the KMP preprocessing phase (building the `next` array) for a pattern P?

(A) $O(1)$

(B) $O(|P|)$

(C) $O(|P|^2 \log |P|)$

(D) $O(|T|\,|P|)$

Answer: (B). The `next` array is built in $O(|P|)$; the brute-force algorithm has no preprocessing.

My answer

Q40. Old exam (String Matching)

The Knuth–Morris–Pratt algorithm that finds the first occurrence of a pattern P of length $|P|$ in a text T of length $|T|$, where $|P| < |T|$, runs in the worst case (tightest bound) in

(A) $O(|P|)$

(B) $O(|T|)$

(C) $O(|T|\,|P|)$

(D) $O(|P| \log |P|)$

Answer: (B). The search phase is linear in the text length, since i never moves back.

My answer

Part B: Programming

Q41. [new]

Write a Java method `int bruteForceSearch(String T, String P)` that returns the index of the first occurrence of pattern `P` in text `T` using the brute-force algorithm, or -1 if there is none.

Answer:

```
public static int bruteForceSearch(String T, String P) {
    int n = T.length(), m = P.length();
    for (int s = 0; s <= n - m; s++) {        // |T| - |P| + 1 alignments
        int j = 0;
        while (j < m && T.charAt(s + j) == P.charAt(j))
            j++;
        if (j == m)
            return s;
    }
    return -1;
}
```

My answer

Q42. [new]

Write a Java method `int bruteForceComparisons(String T, String P)` that runs the brute-force algorithm and returns the number of character comparisons made until the first occurrence is found (or until the search fails). For example, it returns 15 for `T = "ACGACACATAGTCACTTGGCA"`, `P = "ACATA"`.

Answer:

```
public static int bruteForceComparisons(String T, String P) {
    int n = T.length(), m = P.length(), count = 0;
    for (int s = 0; s <= n - m; s++) {
        int j = 0;
        while (j < m) {
            count++;
            if (T.charAt(s + j) != P.charAt(j))
                break;
            j++;
        }
        if (j == m)
            return count;   // first occurrence found
    }
    return count;           // pattern not found
}
```

My answer

Q43. [new]

Write a Java method `List<Integer> bruteForceFindAll(String T, String P)` that returns the starting indexes of all (overlapping) occurrences of `P` in `T`. For `T = "ABABABA"` and `P = "ABA"` it returns `[0, 2, 4]`.

Answer:

```
public static List<Integer> bruteForceFindAll(String T, String P) {
    List<Integer> result = new ArrayList<>();
    int n = T.length(), m = P.length();
    for (int s = 0; s <= n - m; s++) {
        int j = 0;
        while (j < m && T.charAt(s + j) == P.charAt(j))
            j++;
        if (j == m)
            result.add(s);
    }
    return result;
}
```

My answer

Q44. [new]

Write a Java method `String border(String s)` that returns the border of `s`: the longest string that is both a proper prefix and a proper suffix of `s` (the empty string if none). For example, `border("ESTATES")` returns `"ES"`.

Answer:

```
public static String border(String s) {
    // try the longest proper prefix first
    for (int k = s.length() - 1; k >= 1; k--)
        if (s.substring(0, k).equals(s.substring(s.length() - k)))
            return s.substring(0, k);
    return "";   // the border is the empty string (lambda)
}
```

My answer

Q45. [new]

Using `border` from the previous question, write `int[] findNextByDefinition(String P)` that builds the KMP `next` array directly from its definition: `next[0] = -1` and, for $j \geq 1$, `next[j]` is the length of the border of `P[0..j-1]`.

Answer:

```
public static int[] findNextByDefinition(String P) {
    int m = P.length();
    int[] next = new int[m];
    next[0] = -1;
    for (int j = 1; j < m; j++)
        next[j] = border(P.substring(0, j)).length();
    return next;
}
```

My answer

Q46. [new]

Write an efficient $O(|P|)$ method `int[] findNext(String P)` that builds the KMP `next` array (`next[0] = -1`, `next[j]` = border length of `P[0..j-1]`). For `"ABCAABC"` it returns $\{-1, 0, 0, 0, 1, 1, 2\}$.

Answer:

```
public static int[] findNext(String P) {
    int m = P.length();
    int[] next = new int[m];
    next[0] = -1;
    if (m > 1) next[1] = 0;
    int i = 2, k = 0;   // k = border length of P[0..i-2]
    while (i < m) {
        if (P.charAt(i - 1) == P.charAt(k)) {   // border can be extended
            k++;
            next[i] = k;
            i++;
        } else if (k > 0) {
            k = next[k];                        // try a shorter border
        } else {
            next[i] = 0;
            i++;
        }
    }
    return next;
}
```

My answer

Q47. [new]

Using `findNext`, write `int kmpSearch(String T, String P)` that returns the index of the first occurrence of `P` in `T` using the KMP algorithm, or -1 if there is none. The text index must never move back.

Answer:

```
public static int kmpSearch(String T, String P) {
    int[] next = findNext(P);
    int i = 0, j = 0;              // text index, pattern index
    while (i < T.length() && j < P.length()) {
        if (j == -1 || T.charAt(i) == P.charAt(j)) {
            i++; j++;
        } else
            j = next[j];           // shift pattern by j - next[j]
    }
```

```
    if (j == P.length())
        return i - j;              // starting index of match
    return -1;                     // no match
}
```

My answer

Q48. [new]

Write `int kmpComparisons(String T, String P)` that runs KMP (first occurrence) and returns the number of character comparisons `T[i]` vs `P[j]` made. Steps where $j = -1$ are not comparisons. For `T = "ABABABABABABABABABC"`, `P = "ABABABC"` it returns 25.

Answer:

```
public static int kmpComparisons(String T, String P) {
    int[] next = findNext(P);
    int i = 0, j = 0, count = 0;
    while (i < T.length() && j < P.length()) {
        if (j == -1) {             // mismatch at j == 0: shift by 1
            i++; j++;
        } else {
            count++;
            if (T.charAt(i) == P.charAt(j)) {
                i++; j++;
            } else
                j = next[j];
        }
    }
    return count;
}
```

My answer

Q49. [new]

Write `List<Integer> kmpFindAllNonOverlapping(String T, String P)` that uses KMP to return the starting indexes of all non-overlapping occurrences of `P` in `T`: after a full match the pattern is shifted by $|P|$ and the search resumes at pattern index 0. For `T = "ABABABA"`, `P = "ABA"` it returns `[0, 4]`.

Answer:

```
public static List<Integer> kmpFindAllNonOverlapping(String T, String P) {
    List<Integer> result = new ArrayList<>();
    int[] next = findNext(P);
    int i = 0, j = 0, m = P.length();
    while (i < T.length()) {
        if (j == -1 || T.charAt(i) == P.charAt(j)) {
            i++; j++;
            if (j == m) {
                result.add(i - m);
                j = 0;          // shift the pattern by |P|
            }
        } else
            j = next[j];
    }
    return result;
}
```

My answer

Q50. [new]

(Application of string matching.) Write `boolean isRotation(String A, String B)` that returns `true` if `B` is a rotation of `A` (e.g. `"CDEAB"` is a rotation of `"ABCDE"`). Use `kmpSearch`: `B` is a rotation of `A` exactly when both have the same length and `B` occurs in `A + A`.

Answer:

```
public static boolean isRotation(String A, String B) {
    if (A.length() != B.length())
        return false;
    return kmpSearch(A + A, B) != -1;
}
```

My answer